

Laboratory 4

Logistic Regression

Mohsen Pourghasemian and Sadaf Joodaki.
emml@dsp.rwth-aachen.de

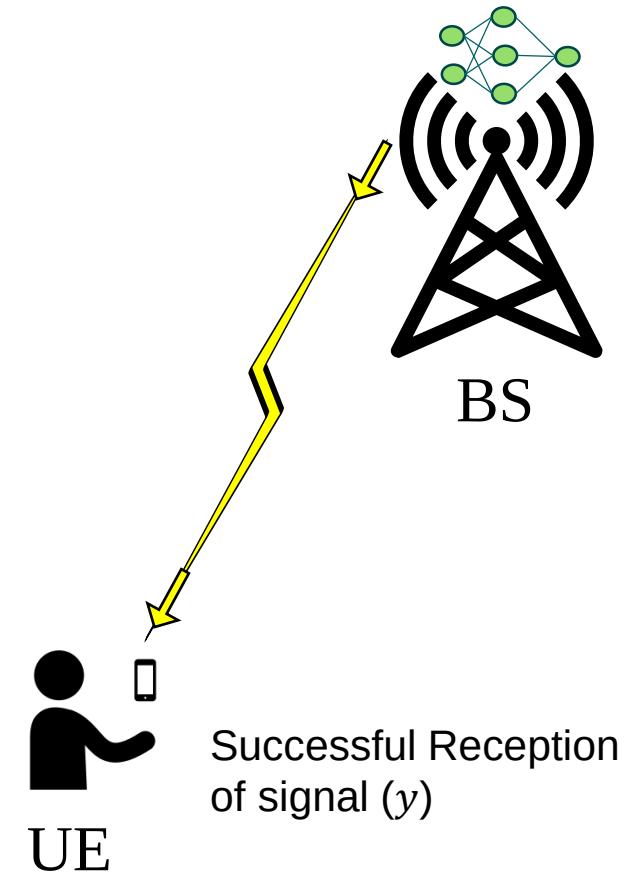
Overview of exercises

- Laboratory 1: K-Means Clustering (1.0 Point)
- Laboratory 2: Grid Search for Linear Regression (1.0 Point)
- Laboratory 3: Gradient Descent for Linear Regression (1.5 Point)
- **Laboratory 4: Logistic Regression (1.0 Point)**
- Laboratory 5: Basics of Neural Network (1.0 Point)
- Laboratory 6: Image Classification with Neural Network (1.5 Points)
- Laboratory 7: Reinforcement learning (1.5 Point)
- Laboratory 8: Deep RL (1.5 Point)

Binary classification: predicting signal reception

- The BS transmits a signal with power x and the UE either receives the signal ($y = 1$) successfully or not ($y = 0$)
- We want to predict whether the UE successfully receives the signal given transmit power x .

Transmit power (x) (in Watt)



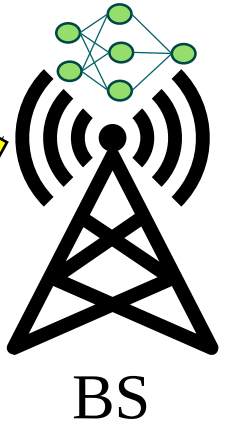
Binary classification: predicting signal reception

- The BS transmits a signal with power x and the UE either receives the signal ($y = 1$) successfully or not ($y = 0$)
- We want to predict whether the UE successfully receives the signal given transmit power x .



Gathering data set with measurement

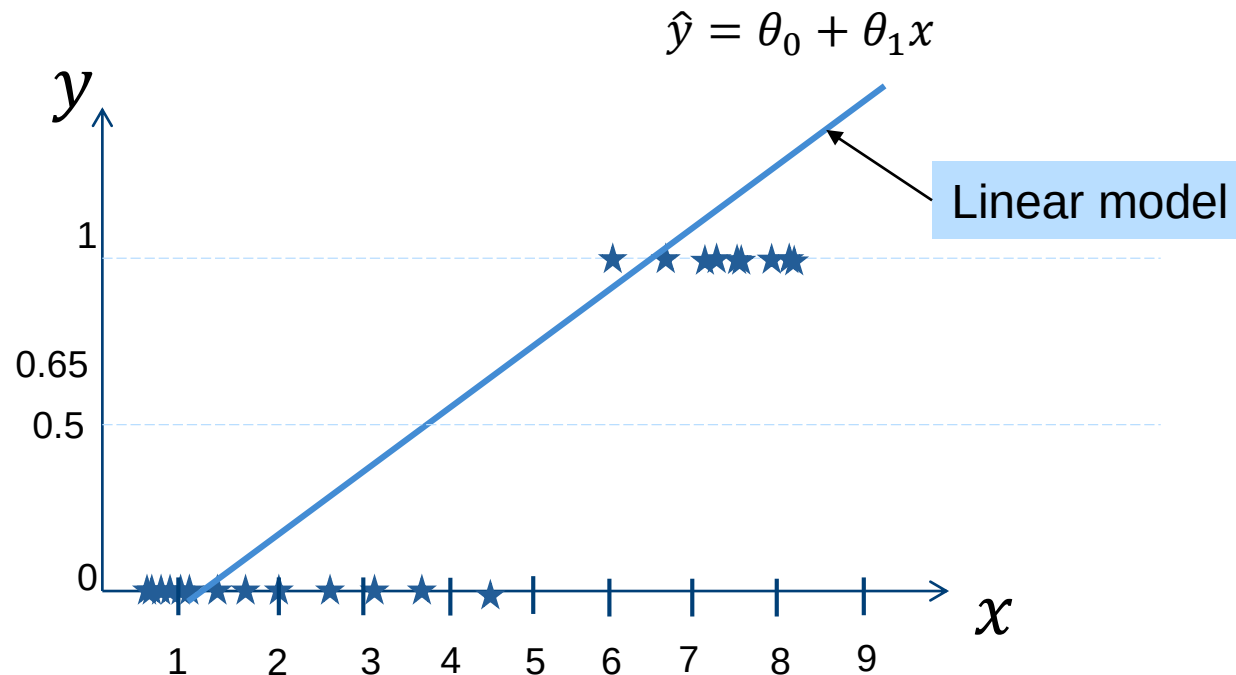
Transmit power (x) (in Watt)



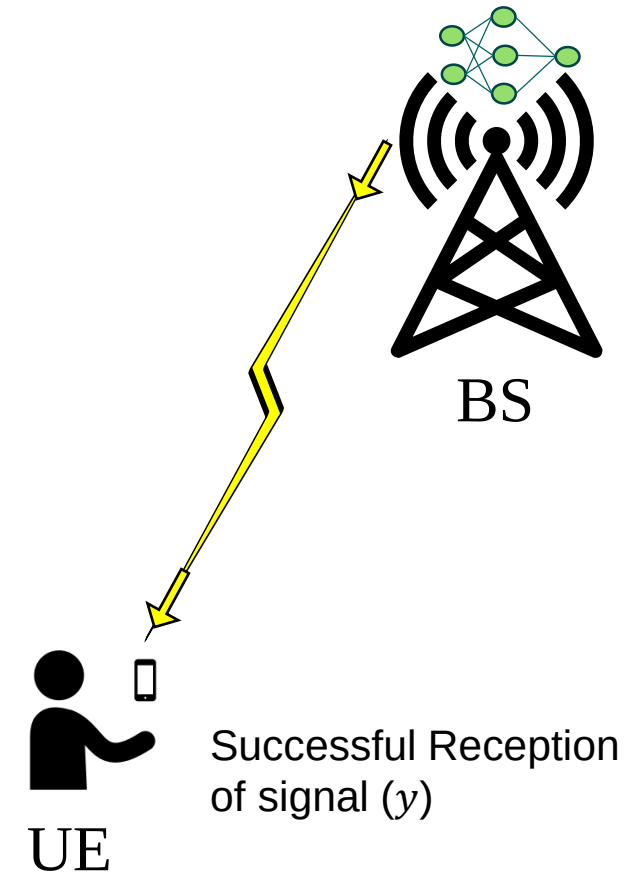
Successful Reception of signal (y)

Linear regression for binary classification

- Prediction with a linear regression model

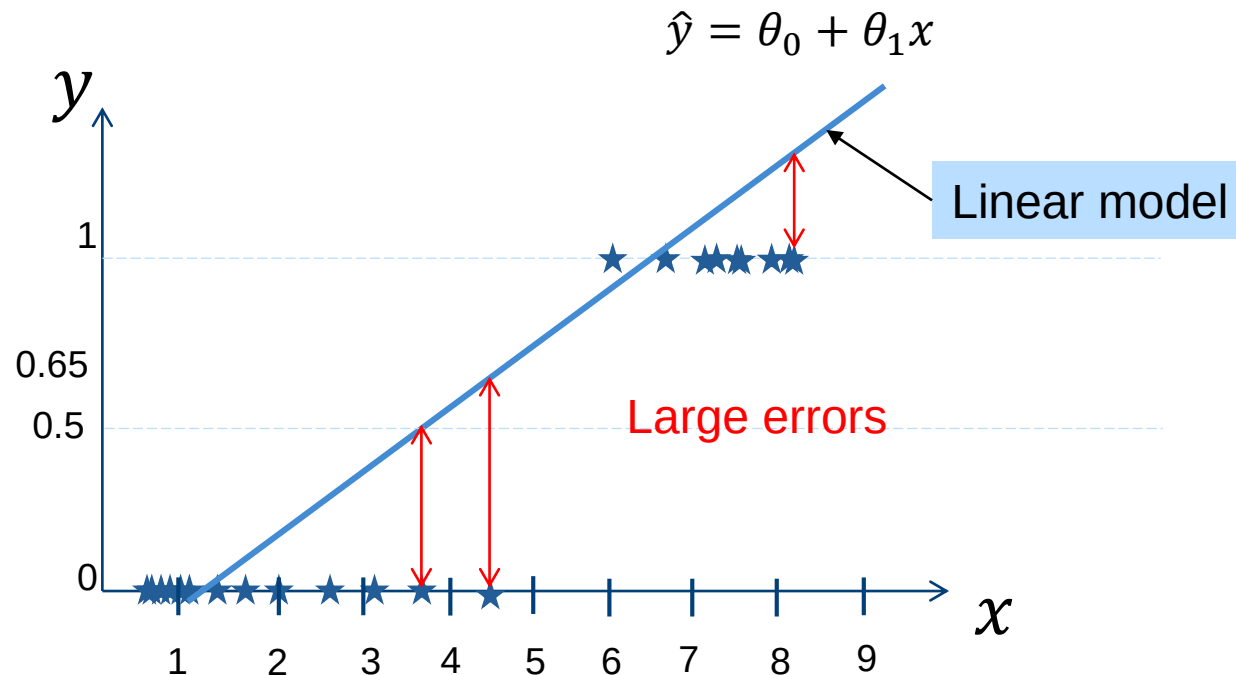


Transmit power (x) (in Watt)

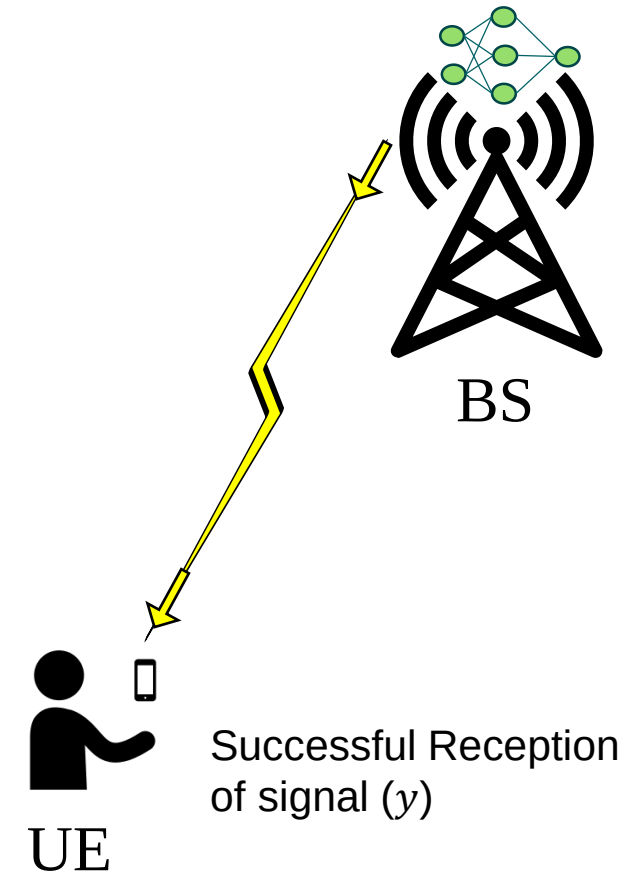


Linear regression for binary classification

- Prediction with a linear regression model



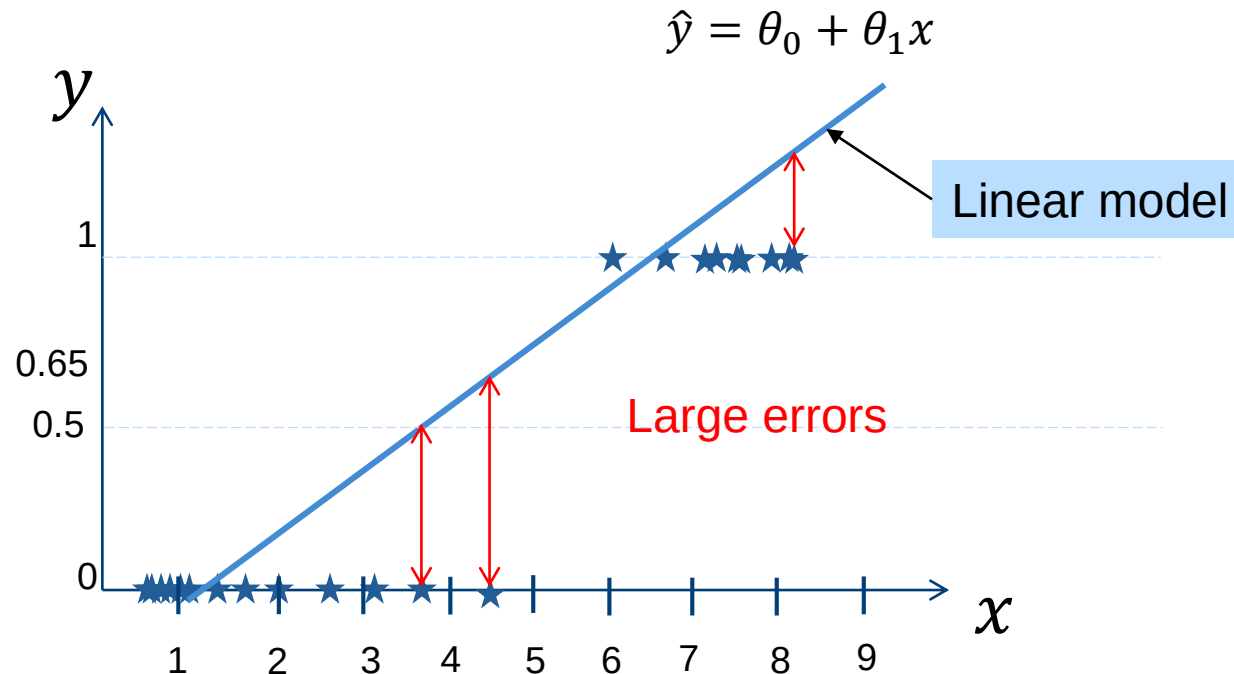
Transmit power (x) (in Watt)



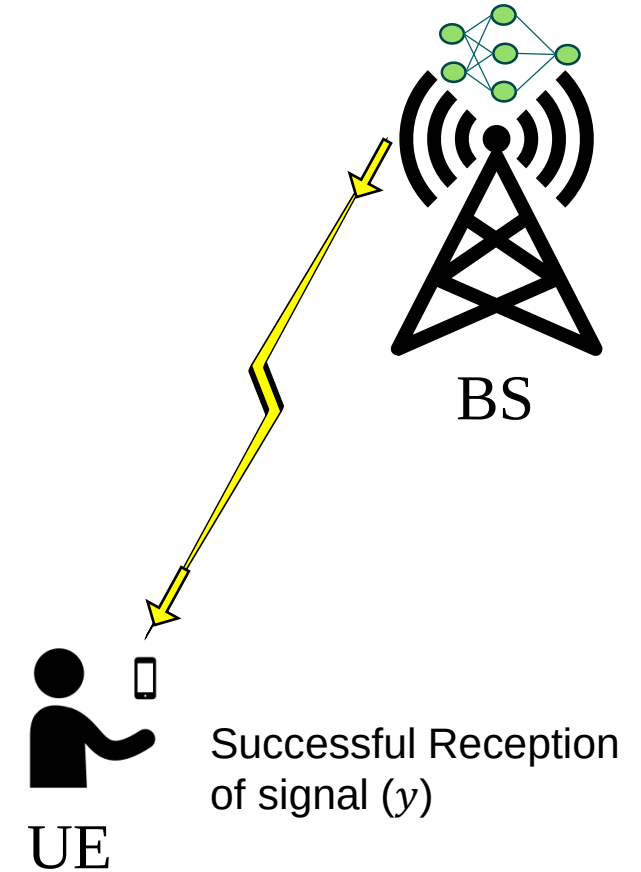
Linear regression for binary classification

- Prediction with a linear regression model

Linear model is not a good model for binary classification

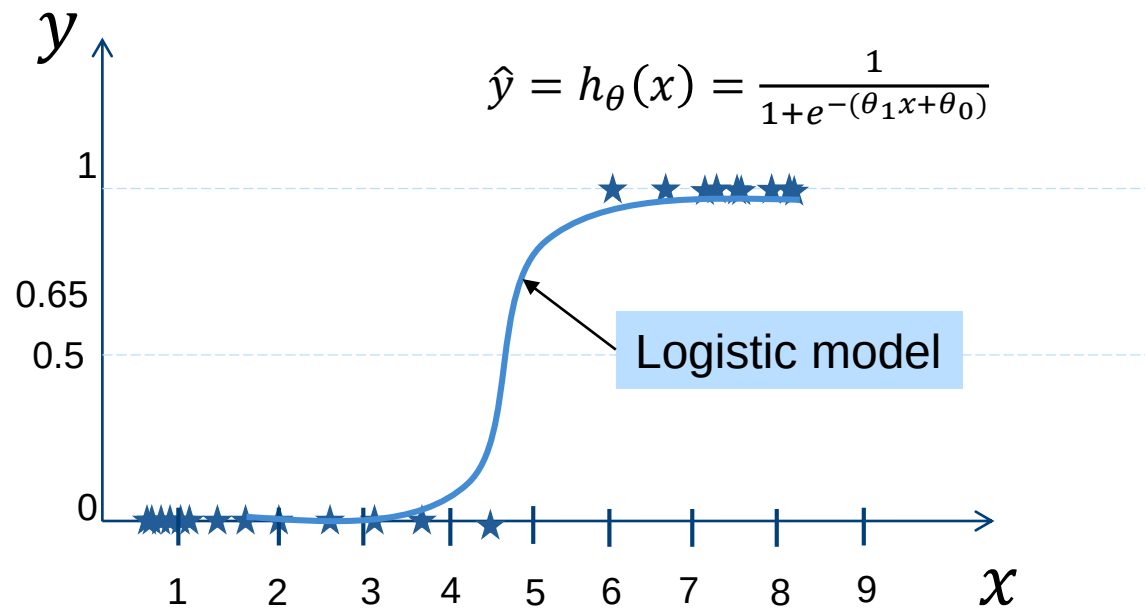


Transmit power (x) (in Watt)



Logistic regression for binary classification

- Uses logistic function (also known as sigmoid function) to model the data



- Sigmoid function: $\sigma(x) = \frac{1}{1+e^{-x}}$
- Applying sigmoid function to linear regression model $\theta_1 x + \theta_0$ yields

$$h_{\theta}(x) = \sigma(\theta_1 x + \theta_0) = \frac{1}{1+e^{-(\theta_1 x + \theta_0)}}$$

Multivariate logistic regression for binary classification

- For each training example, If we have j features, then

$$\theta_0 + \theta_1 x_1 + \dots + \theta_j x_j = \boldsymbol{\theta}^T \mathbf{x}$$

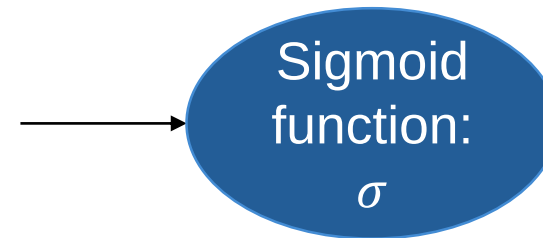
$$\mathbf{x} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ \vdots \\ x_j \end{bmatrix}, \quad \boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \vdots \\ \theta_j \end{bmatrix}$$

Multivariate logistic regression for binary classification

- For each training example, If we have j features, then

$$\theta_0 + \theta_1 x_1 + \dots + \theta_j x_j = \boldsymbol{\theta}^T \mathbf{x}$$

$$\mathbf{x} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ \vdots \\ x_j \end{bmatrix}, \quad \boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \vdots \\ \theta_j \end{bmatrix}$$



$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}}$$

Multivariate logistic regression

Logistic regression model training

Gradient Descent for Logistic Regression

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}} = \frac{1}{1 + e^{-(\theta_0 + x_1 \theta_1 + x_2 \theta_2 + \dots)}} \quad \text{a}$$

Log loss (also know as, logistic loss and cross entropy loss) for single training example ($m = 1$):

$$J(\boldsymbol{\theta}) = -[y \log(h_{\boldsymbol{\theta}}(\mathbf{x})) + (1 - y) \log(1 - h_{\boldsymbol{\theta}}(\mathbf{x}))]$$

Gradient Descent for Logistic Regression

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}} = \frac{1}{1 + e^{-(\theta_0 + x_1 \theta_1 + x_2 \theta_2 + \dots)}} \quad \text{a}$$

Log loss (also known as, logistic loss and cross entropy loss) for single training example ($m = 1$):

$$J(\boldsymbol{\theta}) = -[y \log(h_{\boldsymbol{\theta}}(\mathbf{x})) + (1 - y) \log(1 - h_{\boldsymbol{\theta}}(\mathbf{x}))]$$

Gradient with respect to θ_j ($m = 1$):

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_j} = \underbrace{\frac{\partial}{\partial \theta_j} [-y \log(h_{\boldsymbol{\theta}}(\mathbf{x}))]}_1 - \underbrace{\frac{\partial}{\partial \theta_j} [(1 - y) \log(1 - h_{\boldsymbol{\theta}}(\mathbf{x}))]}_2$$

Gradient Descent for Logistic Regression

Calculation of ①:

$$-y \frac{\partial}{\partial \theta_j} [\log(h_{\theta}(\mathbf{x}))]$$

②

$$\overbrace{h_{\theta}(\mathbf{x}) = \mathbf{u}} \longrightarrow -y \frac{\partial(\log(\mathbf{u}))}{\partial \mathbf{u}} \times \frac{\partial \mathbf{u}}{\partial \theta_j} = \frac{-y}{\mathbf{u}} \times \frac{\partial \mathbf{u}}{\partial \theta_j}$$

Gradient Descent for Logistic Regression

Calculation of 1:

$$-y \frac{\partial}{\partial \theta_j} [\log(h_{\theta}(\mathbf{x}))]$$

b

$$\overbrace{h_{\theta}(\mathbf{x}) = \mathbf{u}} \longrightarrow -y \frac{\partial(\log(\mathbf{u}))}{\partial \mathbf{u}} \times \frac{\partial \mathbf{u}}{\partial \theta_j} = \frac{-y}{\mathbf{u}} \times \frac{\partial \mathbf{u}}{\partial \theta_j}$$

Putting a and b together:

$$-\frac{y}{\frac{1}{1+e^{-\theta^T \mathbf{x}}}} \times \frac{d\left(\frac{1}{1+e^{-\theta^T \mathbf{x}}}\right)}{d\theta_j} = -y \left(1 + e^{-\theta^T \mathbf{x}}\right) \times \frac{x_j e^{-\theta^T \mathbf{x}}}{\left(1+e^{-\theta^T \mathbf{x}}\right)^2} = -\frac{yx_j e^{-\theta^T \mathbf{x}}}{1+e^{-\theta^T \mathbf{x}}} \quad 1$$

Gradient Descent for Logistic Regression

Calculation for 2

$$\begin{aligned} & -(1 - y) \times \frac{1}{1 - \frac{1}{1 + e^{-\theta^T \mathbf{x}}}} \times \frac{-x_j e^{-\theta^T \mathbf{x}}}{(1 + e^{-\theta^T \mathbf{x}})^2} = \\ & (1 - y) \times \frac{1 + e^{-\theta^T \mathbf{x}}}{e^{-\theta^T \mathbf{x}}} \times \frac{x_j e^{-\theta^T \mathbf{x}}}{(1 + e^{-\mathbf{x}^T \theta})^2} = \frac{(1 - y)x_j}{1 + e^{-\mathbf{x}^T \theta}} = 2 \end{aligned}$$

Gradient Descent for Logistic Regression

Calculation for ②

$$-(1 - y) \times \frac{1}{1 - \frac{1}{1 + e^{-\theta^T \mathbf{x}}}} \times \frac{-x_j e^{-\theta^T \mathbf{x}}}{(1 + e^{-\theta^T \mathbf{x}})^2} =$$

$$(1 - y) \times \frac{1 + e^{-\theta^T \mathbf{x}}}{e^{-\theta^T \mathbf{x}}} \times \frac{x_j e^{-\theta^T \mathbf{x}}}{(1 + e^{-\theta^T \mathbf{x}})^2} = \frac{(1 - y)x_j}{1 + e^{-\theta^T \mathbf{x}}} = \textcircled{2}$$

Putting ① and ② together

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_j} = \left(\frac{1}{1 + e^{-\theta^T \mathbf{x}}} - y \right) x_j = (h_{\boldsymbol{\theta}}(\mathbf{x}) - y)x_j$$

In case of θ_0 , we have $x_0 = 1$, then: $\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_0} = (h_{\boldsymbol{\theta}}(\mathbf{x}) - y)$

Update of parameters using multiple training example

Batch Gradient Descent (BGD) – update of θ_j

$$J(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}))]$$

$$\theta_j^{new} = \theta_j^{old} - \alpha \left(\frac{dJ(\boldsymbol{\theta})}{d\theta_j} \right)$$

Where

$$\frac{dJ(\boldsymbol{\theta})}{d\theta_j} = \frac{1}{m} \sum_{i=1}^m (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

Note: θ_j is the coefficient for j -th feature, and $x_j^{(i)}$ is the j -th feature value of the i -th training example.

Batch Gradient Descent (BGD) – update of θ_0

$$\theta_0^{new} = \theta_0^{old} - \alpha \left(\frac{dJ(\boldsymbol{\theta})}{d\theta_0} \right)$$

where

$$\frac{dJ(\boldsymbol{\theta})}{d\theta_0} = \frac{1}{m} \sum_{i=1}^m (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)})$$

Stochastic Gradient Descent (SGD)

- The SGD for logistic regression, using i – th training example ($m = j$):

$$\theta_j^{new} = \theta_j^{old} - \alpha(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})x_j^{(i)}$$

$$\theta_0^{new} = \theta_0^{old} - \alpha(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})$$

Example: Prediction of reception of a signal in a wireless system

- Consider three training examples given in the table below, where the input variable
- (x) indicates the transmit power level chosen by a base station (BS) and the binary variable
- (y) indicates whether a user equipment (UE) is able to successfully detect the signal ($y = 1$) or not ($y = 0$).

	Example 1	Example 2	Example 3
$x(\text{watt})$	9 W	7 W	3 W
y	1	1	0

Prediction of reception of a signal in a wireless system

- By assuming $\theta_0 = -10$ and $\theta_1 = 1.0$ then:
- Write the hypothesis function and calculate the predicted probability $\hat{y}$ for a given x ;
- Calculate the cost for the training examples given in the table;
- Use BGD algorithm and update θ_1 and θ_0 with learning rate $\alpha = 0.01$ (only for one iteration).

Solution

- Calculate **a**:

$$h_{\theta}(\mathbf{x}) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x)}} = \frac{1}{1 + e^{-(-10 + 1.0x)}}$$

	Example 1	Example 2	Example 3
x [W]	9 W	7 W	3 W
$\hat{y}$ (Probability of reception)	0.268	0.047	0.0009

Calculate the cost function

- Calculate **b**: we use the cost function for binary classification as follows

$$J(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\boldsymbol{\theta}}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\boldsymbol{\theta}}(x^{(i)}))] =$$
$$-\frac{1}{3} [\log(0.268) + \log(0.047) + \log(1 - 0.0009)] = 1.458$$

Update the parameters using BGD

- Calculate **c**:

$$\theta_1^{new} = \theta_1 - \alpha \times \left(\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_1} \right) = \theta_1 - \alpha \times \left(\frac{1}{3} \sum_{i=1}^3 (h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)}) \cdot x^{(i)} \right) = 1.0 - 0.01 \times \left(\frac{1}{3} [(0.268 - 1) \times 9 + (0.047 - 1) \times 7 + (0.0009) \times 3] \right) = 1.0 + 0.044 = 1.044$$

$$\theta_0^{new} = \theta_0 - \alpha \times \left(\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_0} \right) = \theta_0 - \alpha \times \left(\frac{1}{3} \sum_{i=1}^3 (h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)}) \right) = -10 - 0.01 \times \left(\frac{1}{3} [(0.268 - 1) + (0.047 - 1) + (0.0009)] \right) = -9.9943$$

Assignment

Logistic Regression for House Price Classification

Objective:

- In this assignment, you will implement a binary logistic regression classifier from scratch using Python, NumPy, and pandas.
- The goal is to classify houses into two classes based on their unit price:
 - label = 1 if the house price is below the mean price
 - label = 0 if the house price is greater than or equal to the mean price
- You will preprocess the dataset, train the logistic regression model using gradient descent, and evaluate the model on training and test data.

Dataset

i	Transaction date (X1)	House age (X2)	Distance (X3)	convenience stores (X4)	Latitude (X5)	Longitude (X6)	house price of unit area (Y)
1	2012.917	32	84.87882	10	24.98298	121.54024	37.9
...	...	...	...	...	...	...	...
414	2013.500	6.5	90.45606	9	24.97433	121.5431	63.9

[Ref.] <https://archive.ics.uci.edu/dataset/477/real+estate+valuation+data+set>

Task

Step 1. Create binary labels

- Compute the mean of all house prices,
(The corresponding function is *make_label(price_list)*)

Step 2. Split the dataset into training and test sets

- Use the first ratio part of the dataset as the training set,
- Use the remaining part as the test set,
(The corresponding function is *train_test_split(df, ratio)*)

Task

Step 3. Normalize the input features

- Use min-max normalization for each feature:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

- Compute min and max only from the training data
- Use the same training min/max values to transform the test data
- If max = min, set the scaled value to 0

(The corresponding functions are *min_max_fit(x)* and *min_max_transform(x, min_list, max_list)*)

- Note: After normalization, we need to add a bias column of ones to the input matrix. This is already done by *add_bias(x)*.

Task

Step 4. Implement the sigmoid function

(The corresponding function is *sigmoid(z)*)

Step 5. Compute the logistic regression cost

- Implement cost $J(\theta)$ for binary classification
- To avoid numerical problems with $\log(0)$, clip probabilities using a small value such as $1e-12$

(The corresponding function is *log_cost(y, p)*)

- Note: For the prediction, we use threshold 0.5; if $p \geq 0.5$, predict label 1, otherwise label 0. This is already done by *predict_label(x, t)*

Step 6. Train logistic regression using gradient descent

- Initialize parameter vector θ with zeros
- Compute predicted probabilities on the training set
- Compute predicted probabilities on the test set
- Compute and store training cost and test cost
- Compute the gradient
- Update the parameters using gradient descent
- Store parameter values during training

(The corresponding function is *train_gd(x_train, y_train, x_test, y_test, lr, steps)*)

Task

Step 7. Model Evaluation:

- Plot the training cost and test cost for each learning rate (0.01, 0.1, 0.5 are given in the code)
- Print:
 - learning rate
 - training accuracy
 - test accuracy

Submission

- Submit your `.py` code on Moodle by **May 25, 2026, before 10:30 AM**.
 - The required libraries, like Numpy, Pandas etc. are provided to you. Make sure you do not use any other libraries or modules to your submitted code, otherwise your grade might be 0.
- ❖ If the code fails to run, it will receive no points.

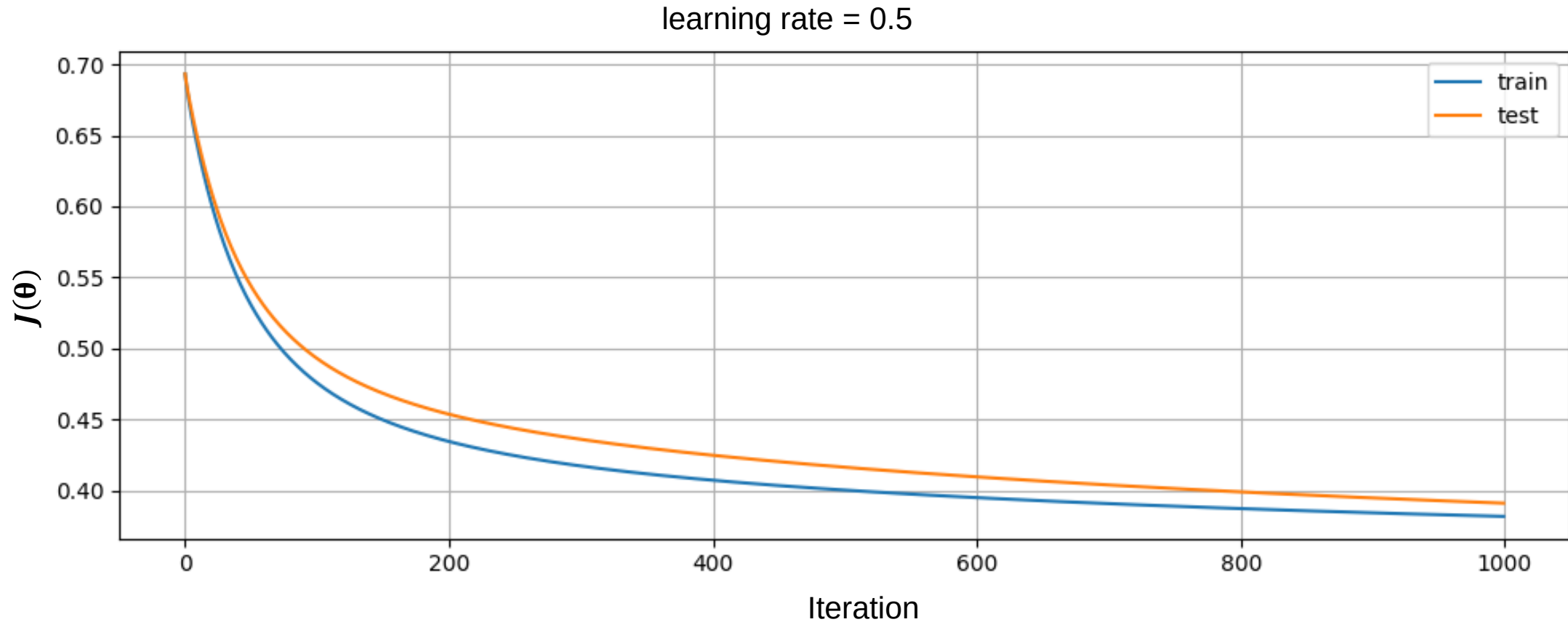
Expected results: Training loss and testing loss

- learning rate: 0.01
- training accuracy: 0.7613
- test accuracy: 0.734

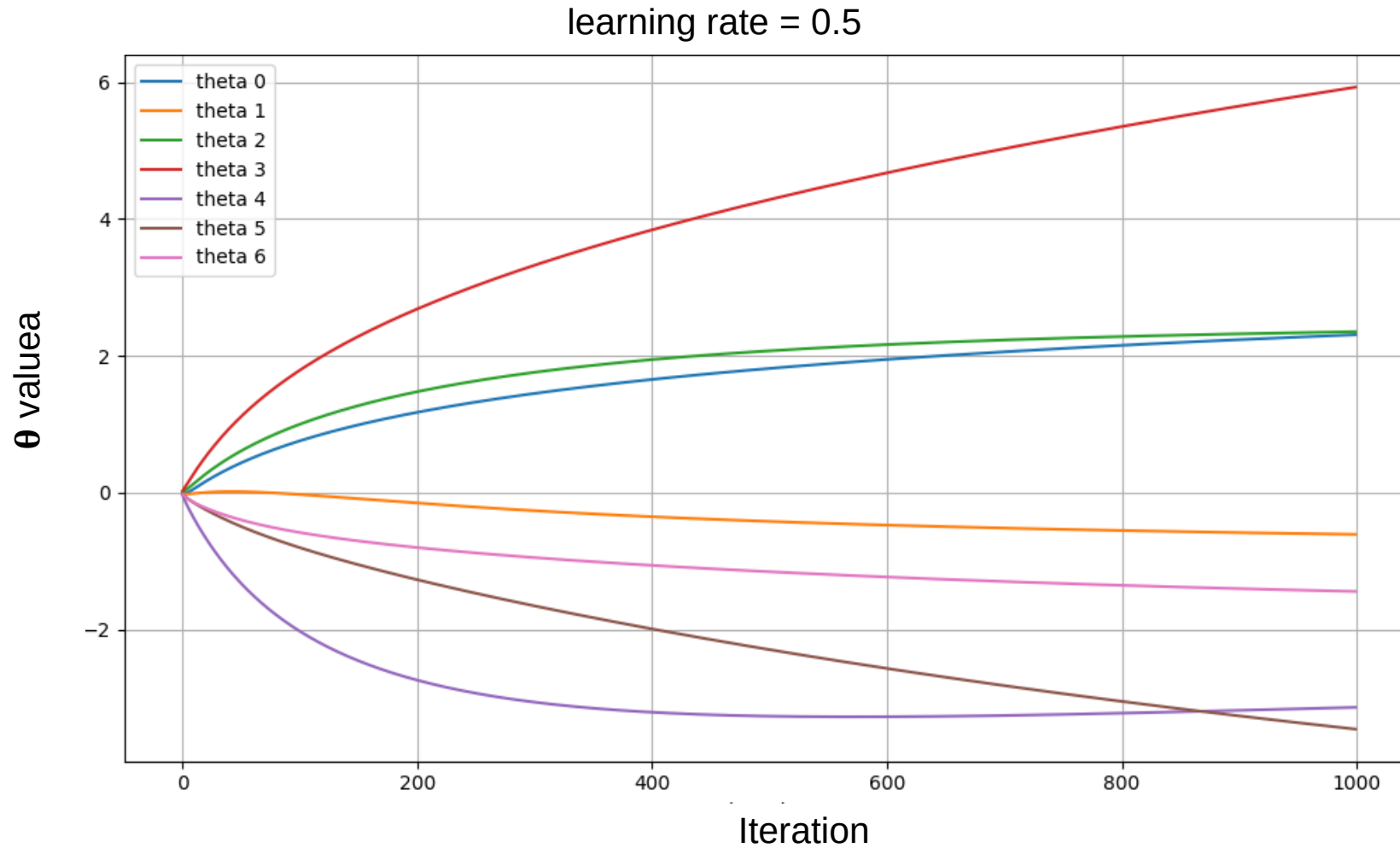
- learning rate: 0.1
- training accuracy: 0.818
- test accuracy: 0.807

- learning rate: 0.5
- training accuracy: 0.842
- test accuracy: 0.807

Expected results: Training and testing cost



Expected results: Convergence of the coefficients vs. iteration



Thank you!

